

LangChain: Converting a Manual Tool Loop to `create_react_agent`

1. What the two original lines were doing

In the manual agent loop, these two lines maintain the conversation history between the AI and the tools:

```
messages.append(ai_message)

messages.append(
    ToolMessage(content=str(observation), tool_call_id=tool_call_id)
)
```

ai_message records what the AI decided to do, such as calling `get_product_price`. **ToolMessage** records the actual result returned by the tool. The `tool_call_id` connects the tool result to the corresponding AI tool call.

2. Manual implementation vs. ReAct agent

The original code manually implements the ReAct-style loop: LLM → tool call → tool execution → tool result → LLM → ... → final answer.

With `create_react_agent`, LangChain/LangGraph manages that loop for you. You provide the model, tools, prompt, and user message.

3. Complete `create_react_agent` version

```
from dotenv import load_dotenv

load_dotenv()

from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langgraph.prebuilt import create_react_agent
from langchain_core.messages import HumanMessage
from langsmith import traceable

MODEL = "qwen3:1.7b"

# =====
# Tools
# =====

@tool
def get_product_price(product: str) -> float:
    """Look up the price of a product in the catalog."""

    print(f"    >> Executing get_product_price(product='{product}')"

    prices = {
        "laptop": 1299.99,
        "headphones": 149.95,
        "keyboard": 89.50,
    }

    return prices.get(product, 0)

@tool
def apply_discount(price: float, discount_tier: str) -> float:
    """Apply a discount tier to a price and return the final price.
    Available tiers: bronze, silver, gold.
    """

    print(
        f"    >> Executing apply_discount("
        f"price={price}, discount_tier='{discount_tier}')"
    )
```

```

    )

    discount_percentages = {
        "bronze": 5,
        "silver": 12,
        "gold": 23,
    }

    discount = discount_percentages.get(discount_tier, 0)

    return round(price * (1 - discount / 100), 2)

# =====
# Agent
# =====

@traceable(name="LangChain ReAct Agent")
def run_agent(question: str):

    # Create the LLM.
    llm = init_chat_model(
        f"ollama:{MODEL}",
        temperature=0,
    )

    # Make the tools available to the agent.
    # We no longer need tools_dict or llm.bind_tools(tools).
    tools = [
        get_product_price,
        apply_discount,
    ]

    # System instructions tell the agent how the tools
    # should be used and in what order.
    system_prompt = """
You are a helpful shopping assistant.

You have access to:
1. A product catalog tool.
2. A discount calculation tool.

STRICT RULES:

1. NEVER guess or assume any product price.
   You MUST call get_product_price first.

2. Only call apply_discount AFTER receiving the price
   from get_product_price.

3. Pass the EXACT price returned by get_product_price
   to apply_discount.

4. NEVER calculate discounts yourself.
   Always use apply_discount.

5. If the user does not specify a discount tier,
   ask the user which tier to use.
   Do NOT assume a tier.

6. After all required tools have been executed,
   provide a clear final answer to the user.
"""

    # create_react_agent manages the ReAct loop:
    #
    #   LLM
    #   ↓
    #   Tool call
    #   ↓
    #   Tool result
    #   ↓
    #   LLM

```

```

#         ↓
#         Tool call
#         ↓
#         ...
#         ↓
#         Final answer
#
# We no longer manually create ToolMessage objects
# or append messages after every tool call.
agent = create_react_agent(
    model=llm,
    tools=tools,
    prompt=system_prompt,
)

print(f"Question: {question}")
print("=" * 60)

# Give the user's question to the agent.
# The agent automatically manages the subsequent
# LLM/tool/LLM loop.
result = agent.invoke(
    {
        "messages": [
            HumanMessage(content=question)
        ]
    }
)

# The result contains the message history.
# The last message normally contains the final AI answer.
final_message = result["messages"][-1]

print(f"\nFinal Answer: {final_message.content}")

return final_message.content

if __name__ == "__main__":

    print("Hello LangChain ReAct Agent!")
    print()

    result = run_agent(
        "What is the price of a laptop after applying a gold discount?"
    )

```

4. What changed?

Manual implementation	create_react_agent
llm.bind_tools(tools)	create_react_agent(model=llm, tools=tools)
llm.invoke(messages)	Agent invokes the LLM internally
ai_message.tool_calls	Agent examines tool calls internally
tools_dict	Agent knows the supplied tools
tool_to_use.invoke(...)	Agent executes tools
messages.append(ai_message)	Agent manages message state
ToolMessage(...)	Agent manages tool messages
for iteration in range(...)	Agent manages the loop
if not tool_calls	Agent detects completion

5. The execution flow

User
|

```

    | "Laptop price after gold discount?"
    v
ReAct Agent
    |
    v
LLM
    |
    | get_product_price("laptop")
    v
get_product_price
    |
    | 1299.99
    v
LLM
    |
    | apply_discount(1299.99, "gold")
    v
apply_discount
    |
    | 1000.49
    v
LLM
    |
    | "The final price is $1,000.49."
    v
User

```

6. The key conceptual shift

Your original code builds the agent loop yourself. `create_react_agent` provides that loop as an abstraction. You still define the tools and the instructions, but the framework handles the repeated tool-calling workflow, message state, tool results, and stopping when the model produces a final response.

7. Important difference from your original code

Your original code explicitly processed only the first tool call per iteration using `tool_calls[0]`. With the ReAct agent, tool-call orchestration is handled by the agent. You should therefore avoid assuming that only one tool call will occur in every iteration.

8. One caveat

`create_react_agent` APIs vary across LangChain/LangGraph releases, and newer LangChain versions also provide newer agent APIs. If the code does not match your installed versions, check the versions of `langchain` and `langgraph` and adjust the import/API accordingly.